

Final Architecture - APB UART IP Core with FIFO and Interrupt

This document defines the finalized architecture for the APB-based UART IP Core project. The goal is to build a reusable UART peripheral using clean RTL architecture, FIFO buffering, interrupt handling, APB protocol integration, and SystemVerilog methodology.

1. Top-Level Architecture

CPU communicates with the UART peripheral through the APB bus.

Main flow:

CPU → APB Wrapper → UART IP Core

UART IP Core contains:

- UART TX
- UART RX
- TX FIFO
- RX FIFO
- Baud Generator
- Interrupt Controller
- Status Registers

2. Major Modules

Module	Functionality
APB Wrapper	Handles APB read/write transactions and register decoding
UART TX	Serial transmitter
UART RX	Serial receiver
TX FIFO	Stores outgoing transmit bytes
RX FIFO	Stores received UART bytes
Baud Generator	Generates baud_tick from clock
Interrupt Controller	Generates IRQ signal
Status Registers	Stores UART and FIFO status

3. Register Map

Address	Register	Purpose
0x00	CONTROL_REG	Enable TX/RX and interrupt enables
0x04	STATUS_REG	FIFO and UART status flags
0x08	BAUD_REG	Baud divider configuration
0x0C	TX_DATA_REG	CPU writes TX data
0x10	RX_DATA_REG	CPU reads RX data

4. Interrupt Architecture

The interrupt controller is implemented as a separate module.

The IRQ signal only informs the CPU that an event occurred.
The CPU determines the exact reason by reading STATUS_REG.

Interrupt events:

- RX data available
- TX completed
- TX FIFO full
- TX FIFO empty
- RX FIFO full
- RX FIFO empty
- FIFO overflow

Final IRQ Logic:

irq = OR of all enabled interrupt events

5. FIFO Architecture

TX FIFO:

- CPU writes data into TX FIFO
- UART TX reads FIFO data during transmission

RX FIFO:

- UART RX writes received bytes into FIFO
- CPU reads received bytes through APB

FIFO status signals:

- full
- empty
- overflow
- underflow

6. SystemVerilog Features

- interface → APB abstraction
- package → register addresses and parameters
- enum → FSM implementation
- assertions → protocol checking
- modport → interface direction control
- struct → transaction grouping

7. Design Methodology

Development Flow:

1. Finalize architecture
2. Define register map
3. Build APB wrapper
4. Implement FIFOs
5. Implement UART TX/RX
6. Add baud generator
7. Add interrupt controller
8. Add assertions

9. Create self-checking testbench
 10. Perform protocol and timing debugging
-

Final Note: This project focuses on reusable RTL architecture, protocol correctness, interrupt handling, FIFO design, APB integration, and scalable SystemVerilog methodology.